

Fact Verification in Large Language Models

Introduction

Fact verification in large language models is the process of checking whether generated statements are supported by reliable evidence. A large language model can produce fluent and confident text even when some details are unsupported, outdated, or factually incorrect. This behavior is commonly discussed as hallucination. In practical applications, fact verification helps determine whether each claim in an answer is grounded in trusted information.

A verification system usually compares a model answer against a knowledge base, retrieved passages, or structured evidence. The goal is not only to judge whether the final answer sounds plausible, but also to decide whether specific claims can be supported. A claim such as "Retrieval-Augmented Generation combines retrieval with generation" may be supported by evidence. A claim such as "RAG was introduced in 2021" should be rejected unless the knowledge base explicitly contains that year.

Fact verification is important for question answering, education, healthcare support, legal assistance, finance, scientific writing, and enterprise search. In these domains, an answer must be judged by evidence rather than language fluency. A retrieval-augmented system can reduce hallucination by retrieving relevant documents before or after generation, but retrieval alone does not guarantee factual correctness. The retrieved evidence must still be checked against the generated claims.

Background

Large language models learn statistical patterns from large text corpora. They can answer many questions because their parameters encode broad linguistic and factual associations. However, the model does not always know whether a statement is supported by a specific source. It may combine related concepts, copy common misconceptions, or generate facts that appear reasonable but are not present in the evidence.

Fact verification research became prominent through datasets and tasks that ask systems to classify claims as supported, refuted, or not enough information. The FEVER benchmark, introduced in 2018, framed fact verification as evidence retrieval followed by claim classification. TruthfulQA, introduced in 2021, evaluated whether models reproduce common false beliefs or provide truthful answers. Later work on hallucination benchmarks extended the focus to generated answers, retrieved evidence, and span-level unsupported content.

In a retrieval-augmented generation system, fact verification is usually performed after retrieval and generation. The system retrieves relevant passages, extracts claims from the answer, compares each claim with the evidence, and assigns a support label. A claim may be fully supported, partially supported, unsupported, or contradicted. This makes the verification process more transparent than assigning one score to the entire answer.

Core Concepts

A claim is a factual statement that can be checked. In a generated answer, claims often appear as sentences, but one sentence can contain multiple claims. For example, "RAG retrieves documents and was introduced in 2020" contains a method claim and a historical year claim. Fact verification works best when claims are separated into smaller units that can be individually compared with evidence.

Evidence is the information used to support or reject a claim. In a RAG system, evidence usually comes from retrieved passages in a local knowledge base. Evidence quality depends on document quality, chunking,

retrieval ranking, and the ability to preserve metadata such as source file, section name, and document title. A noisy evidence passage can reduce verification accuracy even when the knowledge base contains the correct information.

Support scoring estimates how strongly a claim is grounded in evidence. A simple approach uses semantic similarity between the claim and evidence. This is useful but incomplete. Two sentences can be semantically similar while disagreeing on a number, date, person, or entity. For example, "RAG was introduced in 2021" and "RAG was introduced in 2020" are highly similar but factually contradictory. A robust verification system must check factual details, not only semantic closeness.

Natural Language Inference, often called NLI, classifies the relationship between a premise and a hypothesis. In fact verification, the evidence is the premise and the claim is the hypothesis. The NLI label can be entailment, contradiction, or neutral. Entailment means the evidence supports the claim. Contradiction means the evidence conflicts with the claim. Neutral means the evidence does not provide enough information to decide.

Architecture and Working

A practical fact verification pipeline begins with a user query and an initial answer generated by a language model. The system retrieves relevant evidence from a knowledge base using dense retrieval, sparse retrieval, or a hybrid method. Dense retrieval uses embeddings to find semantically related passages. Sparse retrieval, such as BM25, uses exact keyword and term frequency signals. Hybrid retrieval combines both methods so that semantic meaning and exact terms are both considered.

After retrieval, the generated answer is divided into claims. Each claim is compared with the top evidence passages. The verifier calculates semantic similarity, lexical overlap, factual consistency, and optional NLI output. Factual consistency checks focus on details such as years, dates, numeric values, named entities, and unsupported technical phrases. This is especially useful because many hallucinations differ from the evidence by only one factual detail.

The detector then assigns a label to each claim. A supported claim has clear evidence. A partially supported claim is related to evidence but may be too broad or incomplete. A potential hallucination is unsupported, contradicted, or dependent on facts that are not found in the evidence. The system can also highlight suspicious spans, such as an unsupported year, an incorrect entity, or a phrase like "fine-tuning" when the evidence only says "uses retrieved passages as context".

The correction stage rewrites the answer using only supported evidence. A safe correction module should not simply generate a new answer. It should preserve supported claims, remove unsupported details, replace incorrect facts with evidence-supported facts, and refuse unsupported claims when evidence is insufficient. The corrected answer should be checked again after rewriting because a language model can accidentally introduce new unsupported information during correction.

Key Components

The knowledge base is the foundation of fact verification. Clean text documents, markdown files, JSON records, and well-extracted PDFs provide reliable evidence. Documents should be preprocessed to remove noise such as URLs, repeated headers, references, broken equations, and irrelevant page artifacts. Clean preprocessing improves chunk quality and reduces false matches during retrieval.

The retriever selects evidence for each query or claim. Dense retrieval is useful when the query and evidence use different wording. BM25 is useful when exact terms, acronyms, years, model names, or benchmark names

are important. Reciprocal Rank Fusion combines retrieval rankings and helps return passages that are strong under more than one retrieval method.

The claim extractor converts long answers into checkable statements. It should avoid treating polite assistant phrases as factual claims. Sentences such as "I could not find enough evidence" or "Please provide more context" are answerability statements, not factual claims about the world. Filtering these phrases prevents misleading hallucination scores.

The support scorer combines multiple signals. Semantic similarity measures meaning overlap. Lexical overlap measures shared words and entities. Factual consistency detects mismatched or unsupported years, numbers, dates, and names. NLI adds a logical verification signal. A strong system does not rely on a single metric because each metric has limitations.

The correction module produces the final grounded answer. It should use retrieved evidence, cite evidence where appropriate, avoid unsupported examples, and explicitly state when the knowledge base does not support a claim. A post-correction verification loop improves reliability by checking whether the corrected answer still contains unsupported claims.

Advantages

Fact verification improves trust by showing which parts of an answer are supported by evidence. Instead of treating an entire answer as correct or incorrect, claim-level verification identifies specific unsupported statements. This is helpful for debugging model behavior and explaining system decisions to users.

A retrieval-grounded verification system can work with local or domain-specific knowledge. This is useful when the language model does not have recent, private, or specialized information in its parameters. The verifier checks answers against the current knowledge base rather than relying only on model memory.

Hybrid retrieval improves evidence quality because it combines semantic understanding with keyword precision. Dense embeddings can find conceptually related passages, while BM25 can preserve exact matches for acronyms, dates, model names, and benchmark names. This is important for technical domains where a small factual detail can change the truth of a claim.

Span-level highlighting makes hallucination detection easier to interpret. If a claim says "RAG was introduced in 2021" but the evidence does not support the year, the system can highlight only "2021". This is more useful than marking the entire sentence as wrong, because it shows exactly which part needs correction.

Limitations

Fact verification depends on the quality and coverage of the knowledge base. If the knowledge base does not contain evidence for a claim, the system should not guess. It should respond that evidence is insufficient. This behavior may appear conservative, but it is safer than inventing unsupported details.

Semantic similarity can produce false support. Two claims may be similar while disagreeing on a date, name, number, or cause. This is why similarity must be combined with factual consistency checks and optional NLI verification. A high similarity score alone is not proof of factual correctness.

NLI models can also make mistakes. Some NLI models classify unrelated claim-evidence pairs as contradiction instead of neutral. A practical system should use topical relevance guards before trusting an NLI contradiction. Contradiction should be accepted only when the claim and evidence are sufficiently related.

PDF extraction can introduce noise. Research papers often contain tables, equations, figures, references, headers, and broken line ordering. These artifacts can reduce retrieval quality. Curated text-heavy documents and clean technical PDFs often produce better knowledge-base chunks than raw academic PDFs.

Real-World Applications

In education, fact verification can help students check whether generated explanations are supported by course material. A tutor system can answer questions using retrieved textbook passages and flag unsupported claims before showing the final answer.

In healthcare support, verification can reduce the risk of unsupported medical explanations. A system can be configured to answer only from approved clinical documents and refuse questions when evidence is missing. This does not replace medical professionals, but it can improve the safety of informational tools.

In legal and compliance workflows, generated summaries must be grounded in source documents. Claim-level verification can help identify whether a contract summary, policy explanation, or regulatory answer is supported by retrieved text.

In enterprise knowledge management, employees often ask questions about internal processes, product documentation, or technical manuals. Retrieval-grounded verification helps ensure that the response is based on company-approved documents rather than model assumptions.

Comparison with Related Methods

A basic language model answer uses only the model parameters at generation time. This approach is fast but can hallucinate because the model may not know the latest or domain-specific facts.

Retrieval-Augmented Generation adds external evidence, but the answer can still include unsupported details if the generation step is not checked.

A similarity-only verifier compares claim embeddings with evidence embeddings. This is lightweight and useful for a first-pass score, but it struggles with factual contradictions involving small changes. A fact-aware verifier adds numeric, date, entity, and phrase-level checks to catch errors that similarity misses.

An NLI-based verifier directly estimates whether evidence entails or contradicts a claim. NLI is powerful for logical support detection, but it can be slower and may mislabel unrelated evidence. A practical system treats NLI as one signal among several rather than the only authority.

A span-level hallucination detector identifies the exact unsupported phrase inside a claim. This improves explainability because users can see whether the problem is a year, name, quantity, or unsupported concept. Span-level detection is especially useful for correction because the system can replace only the unsupported detail.

Challenges

One challenge is evidence selection. If retrieval returns a weak or unrelated passage, the detector may produce a poor judgment even if the knowledge base contains the correct answer elsewhere. This can be improved by hybrid retrieval, reranking, metadata-aware scoring, and better chunking.

Another challenge is claim granularity. Long sentences may contain several facts. If the detector treats the entire sentence as one claim, it may miss that one part is supported while another part is unsupported. Sentence splitting, clause splitting, and span highlighting improve precision.

Correction is also difficult. A language model can accidentally add new claims while rewriting an answer. A post-correction verification loop is important because the corrected answer must be checked again. The safest correction policy is to remove or explicitly refuse unsupported factual details.

Evaluation requires labeled examples. A useful benchmark should include queries, generated answers, retrieved evidence, expected supported claims, expected unsupported claims, and gold hallucinated spans.

Without labeled evaluation, it is difficult to measure precision, recall, F1 score, faithfulness, and factual improvement.

Future Directions

Future fact verification systems can use adaptive retrieval depth. Simple queries may need only a few passages, while complex or disputed queries may require more evidence. Retrieval depth can be adjusted based on confidence, answerability, and disagreement between verification signals.

Confidence calibration is another important direction. A system should not only label a claim as supported or unsupported, but also estimate how reliable that label is. Calibrated confidence helps users interpret borderline cases and decide when human review is needed.

Stronger NLI models, domain-specific verifiers, and lightweight rerankers can improve factual accuracy. However, these components must be balanced against hardware limits. For local systems running on modest laptops, optional verification layers and cached embeddings are practical engineering choices.

Multimodal fact verification is also important. Future systems may verify claims using text, images, charts, tables, and structured databases. This requires stronger document parsing and evidence grounding across different data formats.

Real-time knowledge updating can make verification systems more useful. When the knowledge base changes, the retrieval index should be rebuilt or updated so that answers reflect the latest approved evidence. This is especially important for enterprise and scientific applications.

Conclusion

Fact verification in large language models is essential for building trustworthy AI systems. Fluency does not guarantee correctness, and retrieval does not automatically guarantee grounded answers. A reliable system must retrieve evidence, extract claims, score support, detect unsupported details, correct unsafe content, and re-check the corrected answer.

A practical RAG-based fact verification system combines dense retrieval, sparse retrieval, factual consistency checks, optional NLI, span highlighting, and evidence-grounded correction. This layered design is stronger than relying on a single similarity score. It can detect contradictions involving years, entities, numbers, unsupported examples, and unsupported technical wording.

The safest behavior is to answer only when evidence supports the claim and to clearly state when evidence is insufficient. This makes the system useful for hallucination detection and correction because it does not merely produce a better-sounding answer. It produces an answer that is explicitly grounded in available knowledge.